State Machines and the LCD Display

Module	ET4047
Date	27/10/2026
Lab No.	Lab 6
Author	Luke Griffin

This week you build **finite state machines** on the Arduino. You start with a traffic light that changes colour on a timer, connect real LEDs to it, turn it into a **pedestrian crossing** that reacts to a button without ever calling `delay()`, and finish with a **code lock** that checks what you type using the **SHA-256** hash from last week.

#	Task	What is new
1	Traffic light on the LCD	enum, a state variable and switch-case
2	Traffic light with LEDs	every state sets every output; text arrays indexed by the state
3	Pedestrian crossing	a button input, and timing with millis() instead of delay()
4	SHA-256 code lock	a state machine that makes its decisions with a hash

1 Setup

Equipment

Item	Qty	Notes
Arduino Nano (classic) **or** Nano Every, on the Uno-style breakout shield	1	All the code works on either board.
16×2 LCD with I2C backpack, or Adafruit RGB LCD shield	1	Either works: the same sketches run on both.
LEDs: one red, one yellow, one green	3	Tasks 2–4. If your shield has its own LEDs you can use those instead.
220 Ω resistors (red-red-brown)	3	One in series with each LED.
Push button	1	Task 3 only.
Breadboard and jumper wires	~12	
USB cable	1	

Software

- Arduino IDE. Select the board you have: *Tools → Board → Arduino AVR Boards → Arduino Nano* (classic), or *Arduino megaAVR Boards → Arduino Nano Every* (install the megaAVR package from the Boards Manager first). Classic Nano clones may also need *Tools → Processor → ATmega328P (Old Bootloader)*.

© 2026 by University of Limerick. Author: Luke Griffin

- **The library for your display only:** LiquidCrystal I2C (by Frank de Brabander) for the I2C backpack LCD, or Adafruit RGB LCD Shield Library (by Adafruit) for the RGB shield. Tools → Manage Libraries… and search for it.
- **The lab code,** `W07_State_Machines_Lab_Code.zip`. Unzip it somewhere you can find it again.

Set the code for your display

Every sketch works with **both** displays. Each one is a single .ino file that opens with File → Open, and the comments in it explain each step. Near the top of every sketch there is one line that says which display you have:

```
// ----- WHICH DISPLAY DO YOU HAVE? (the only line you may need to change) -----
//   0 = 16x2 LCD with I2C backpack  (a separate screen with four wires)
//   1 = Adafruit RGB LCD shield     (plugs on and has five buttons)
#define RGB_SHIELD 0
```

Display	How to recognise it	Set
16×2 LCD with I2C backpack	A separate screen with a small board on the back and four wires	`#define RGB_SHIELD 0`
Adafruit RGB LCD shield	Plugs straight onto the breakout board and has five buttons	`#define RGB_SHIELD 1`

The sketches come set to **0** (I2C backpack). If you have the RGB shield, change it to **1** in each sketch before uploading. On the RGB shield the backlight also changes colour to match the light.

Task	Sketch	Hardware
1	`Task1_Traffic_Light`	LCD
2	`Task2_Traffic_Light_LEDs`	LCD + 3 LEDs
3	`Task3_Pedestrian_Crossing`	LCD + 3 LEDs + button on D2
4	`Task4_SHA256_Lock`	LCD + 3 LEDs, codes typed in the Serial Monitor

LCD wiring (all tasks)

I2C LCD connections

From		To	Purpose
LCD **GND**	→	Nano **GND**	Ground
LCD **VCC**	→	Nano **5V**	Power
LCD **SDA**	→	Nano **A4**	I2C data
LCD **SCL**	→	Nano **A5**	I2C clock

© 2026 by University of Limerick. Author: Luke Griffin

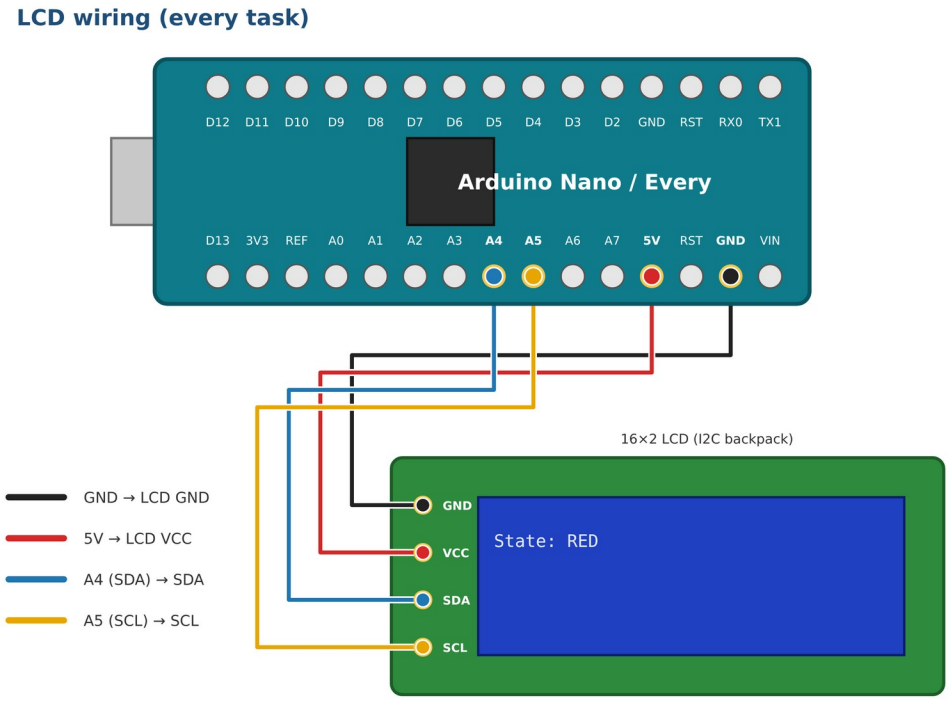


Figure 1: LCD wiring. The pinout is the same on the classic Nano and the Nano Every.

Using the Adafruit RGB LCD shield instead? Plug it onto the Uno-style breakout shield. It uses the same I2C pins (SDA on A4, SCL on A5), so no wiring is needed.

2 What is a state machine?

A **finite state machine** (FSM) is a way to organise behaviour into a fixed number of **states**. At any moment the system is in exactly **one** state. Something, such as a timer running out or a button being pressed, causes a **transition** to another state. Instead of one long sequence of commands or a tangle of if-else statements, you describe each state on its own and the arrows between them. That makes the code easier to read, to change and to debug: you can look at one state at a time.

A traffic light is the classic example. It is always RED, GREEN or YELLOW, never two at once, and it moves RED → GREEN → YELLOW → RED on a timer. Every state machine in this lab is written with the same three parts:

Part	In the code	What it does
1. The states	`enum TrafficLightState { RED, GREEN, YELLOW };`	Gives each state a name. The compiler numbers them 0, 1, 2 for you.
2. The current state	`TrafficLightState currentState = RED;`	Remembers which state the system is in. It also sets the starting state.
3. The behaviour	`switch (currentState) { case RED: … }`	Runs the code for the current state only, and decides when to move on.

© 2026 by University of Limerick. Author: Luke Griffin

Why an enum and not just numbers?

`case RED:` is easier to read than `case 0:`, and the compiler treats `TrafficLightState` as its own type, so it can warn you about mistakes. You can still use the number behind a name when you need it: Task 2 uses it to look up the text to show on the LCD.

3 Task 1: A traffic light on the LCD

Objective: run a three-state traffic light, see how the enum, the state variable and the switch-case work together, and change the state machine yourself. Only the LCD is needed: the state is shown as text.

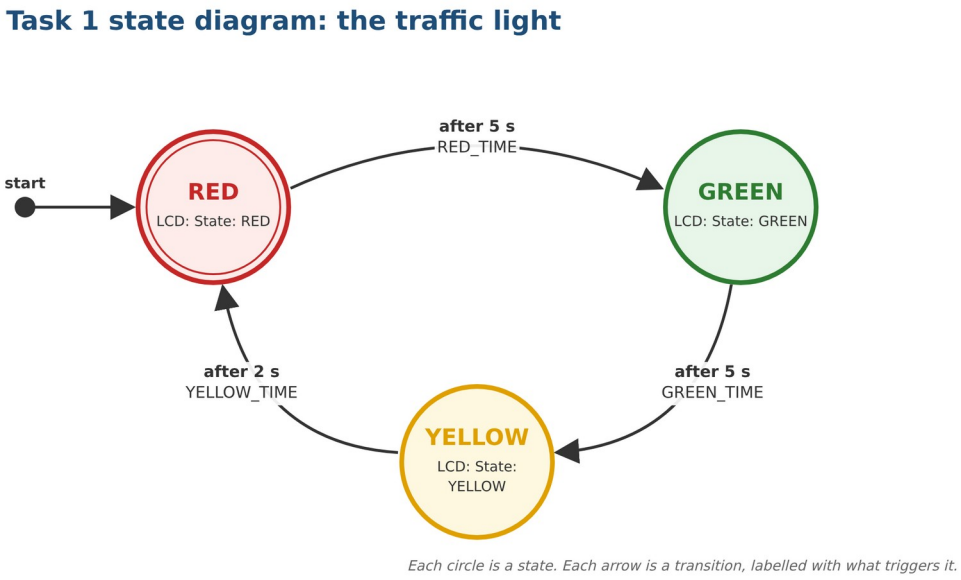


Figure 2: Task 1 state diagram. The double circle is the starting state.

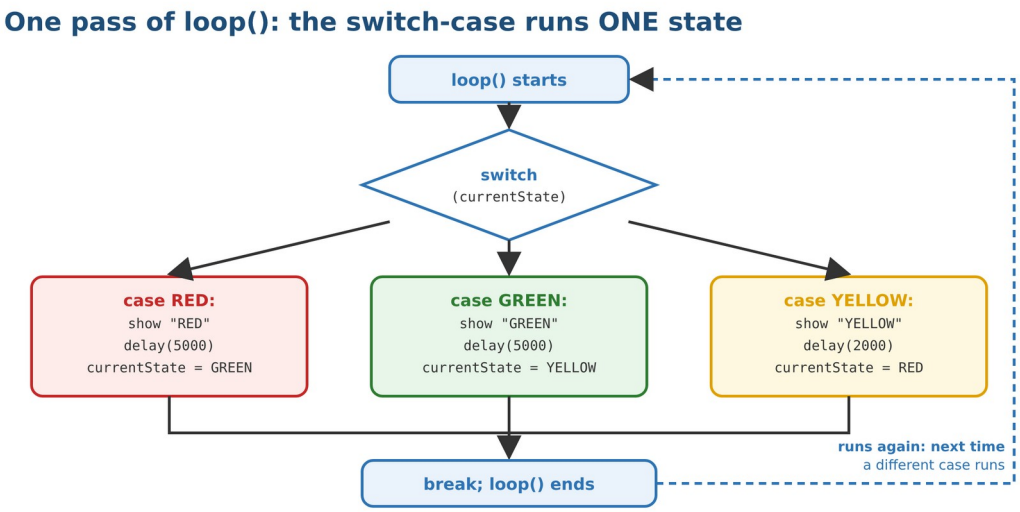


Figure 3: Each pass of loop() runs one case, which changes currentState so the next pass runs the next case.

© 2026 by University of Limerick. Author: Luke Griffin

What happens

> **Code**
>
> Upload `Task1_Traffic_Light` (with `RGB_SHIELD` set for your display) and open the Serial Monitor at 9600 baud.

The sketch starts in RED. Each time `loop()` runs, the switch-case picks the case for the current state (Figure 3). That case writes the state name on the LCD, waits with `delay()`, and then changes `currentState` to the next state. When `loop()` runs again, a different case runs.

Expected result: the LCD shows `State: RED` for 5 s, `State: GREEN` for 5 s, `State: YELLOW` for 2 s, and repeats.

Task 1 questions

1a Run it

Upload the code and check the states change in the right order. **(record)** Take a photo of the LCD.

1b The transition table

A state diagram (Figure 2) can also be written as a table. Fill it in from the code.

Current state	Time in state	Next state	LCD shows
RED			
GREEN			
YELLOW			

How long does one full cycle take?

Your answer:

1c Change the timing

Make YELLOW last 3 seconds instead of 2. Which line did you change? Why is it better to keep the times in named constants such as `YELLOW_TIME` than to type `delay(2000)` inside the case?

Your answer:

© 2026 by University of Limerick. Author: Luke Griffin

1d Add a state

In the UK, the light shows red and amber together before going green. Add a fourth state, `RED_AMBER`, which shows `State: RED+AMBER` for 2 s, so the cycle becomes RED → RED_AMBER → GREEN → YELLOW → RED. You need to change three places in the code. Which three? **(record)** Draw your new state diagram.

Your answer:

4 Task 2: Traffic light with real LEDs

Objective: drive a red, a yellow and a green LED from the state machine. Every state sets **all three** LEDs, so exactly one is on at any time, and the LCD shows the state and what it means for a driver.

Wiring

Keep the LCD wiring from Section 1. Each LED needs a 220 Ω resistor in series, or too much current flows and the LED or the pin can be damaged.

LED connections (plus the LCD wiring from Section 1)

From		To	Purpose
Nano **D5**	→	220 Ω → **red** LED long leg	Red light
Nano **D6**	→	220 Ω → **yellow** LED long leg	Yellow light
Nano **D7**	→	220 Ω → **green** LED long leg	Green light
Each LED short leg	→	GND rail → Nano **GND**	Common ground for all three LEDs

© 2026 by University of Limerick. Author: Luke Griffin

Task 2: three LEDs on D5, D6, D7

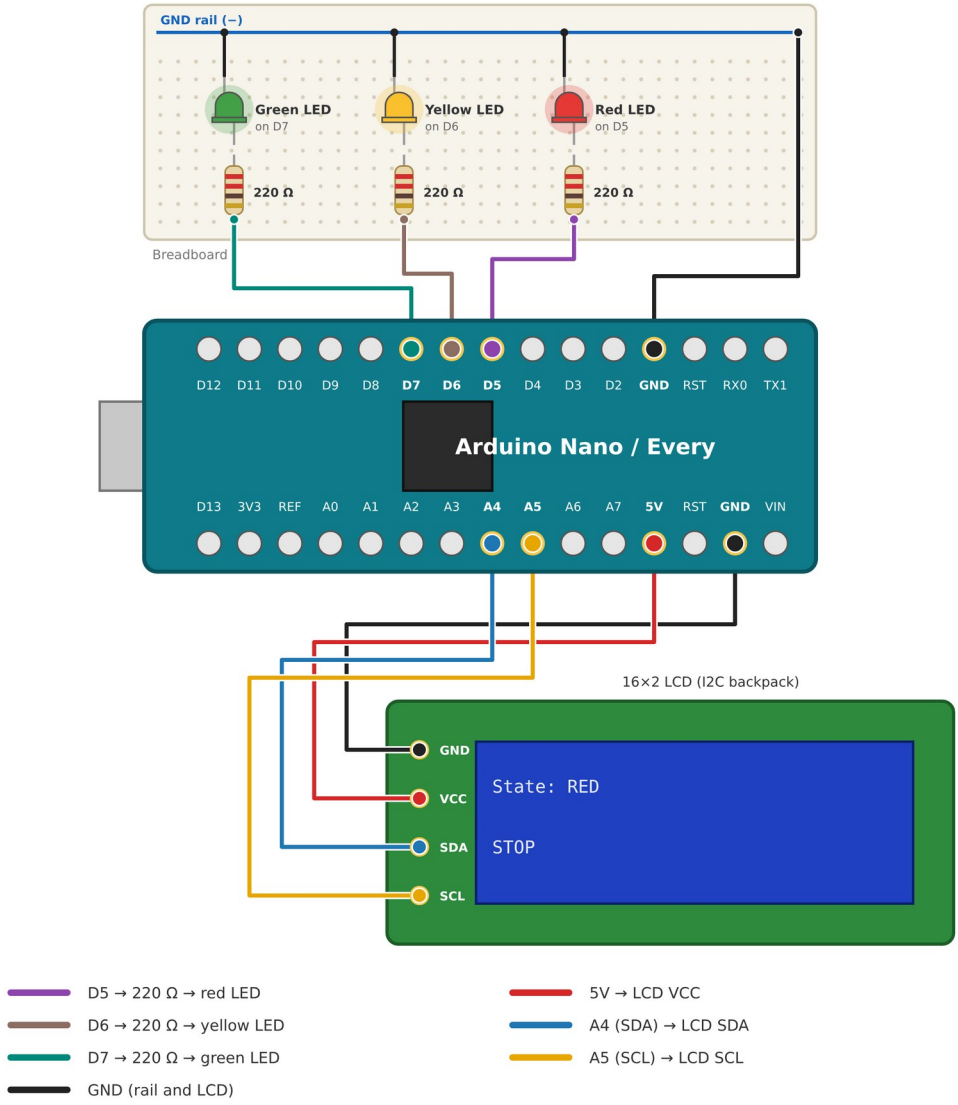


Figure 4: Task 2 wiring: three LEDs, each with a 220 Ω resistor, plus the LCD

Which way round does an LED go?

The **long leg** (anode, +) goes towards the Arduino pin, through the resistor. The **short leg** (cathode, −), on the side with the flat edge, goes to GND. An LED that is in backwards does not light, but it is not damaged.

LEDs on your shield? If your breakout shield has its own LEDs, you can use them instead: change `RED_PIN`, `YELLOW_PIN` and `GREEN_PIN` at the top of the sketch to match.

What happens

Code

Upload `Task2_Traffic_Light_LEDs` (with `RGB_SHIELD` set for your display).

The state machine is the same as Task 1. Each case now starts with `setLights(…)`, which sets all three LEDs in one call, for example `setLights(HIGH, LOW, LOW)` for RED. The text on the LCD comes from two arrays that are in the same order as the enum, so `stateNames[currentState]` gives `"RED"`, `"GREEN"` or `"YELLOW"`.

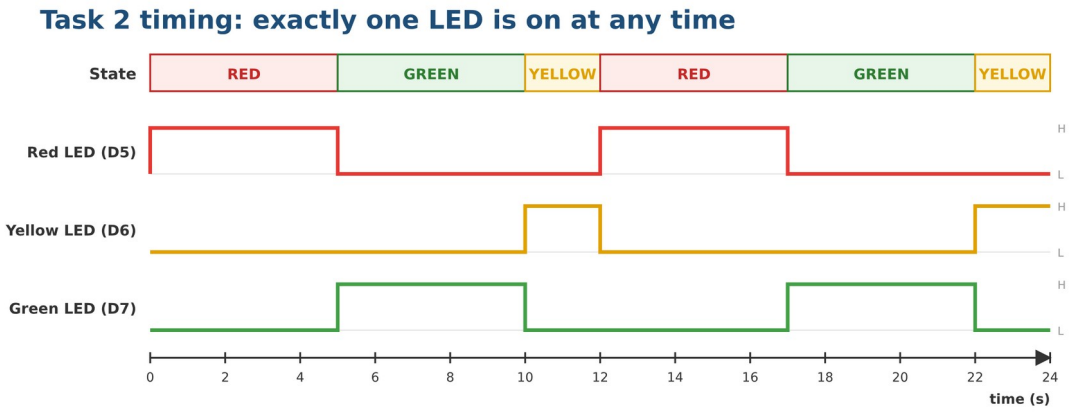


Figure 5: The three LED outputs over two cycles: in every state exactly one is HIGH

Task 2 questions

2a Run it

Upload the code and check the LEDs and the LCD change together. **(record)** Paste a few lines from the Serial Monitor. They should look like this:

Serial Monitor

```
Traffic light ready
State 0: RED
State 1: GREEN
State 2: YELLOW
```

2b Forget to switch an LED off

In `case GREEN:`, replace `setLights(LOW, LOW, HIGH);` with `digitalWrite(GREEN_PIN, HIGH);` and do the same with the matching pin in the other two cases. Run it for two full cycles. What do the LEDs show, and why? Compare with Figure 5. Then put `setLights()` back.

Your answer:

2c The numbers behind the names

The Serial Monitor prints the number behind each state name. What would the LCD show if you changed the enum to `{ RED, YELLOW, GREEN }` but left the two text arrays as they are? Why?

© 2026 by University of Limerick. Author: Luke Griffin

Your answer:

2d The resistor

A red LED drops about 2 V when it is on, and the pin gives 5 V. Use Ohm's law to work out roughly how much current flows through a 220 Ω resistor. (An Arduino pin should not supply more than about 20 mA.)

Your answer:

5 Task 3: A pedestrian crossing

Objective: make the state machine react to an **input**. The cars' light stays GREEN until a pedestrian presses the button; then it goes YELLOW, then RED while the pedestrians WALK (with a countdown on the LCD), stays RED for a moment to CLEAR the crossing, and goes back to GREEN.

Wiring

Add the button (keep the LEDs and LCD from Task 2)

From		To	Purpose
Nano **D2**	→	Push button, one leg	Pedestrian button input
Push button, other leg	→	GND rail	Pressing the button pulls D2 to 0 V

© 2026 by University of Limerick. Author: Luke Griffin

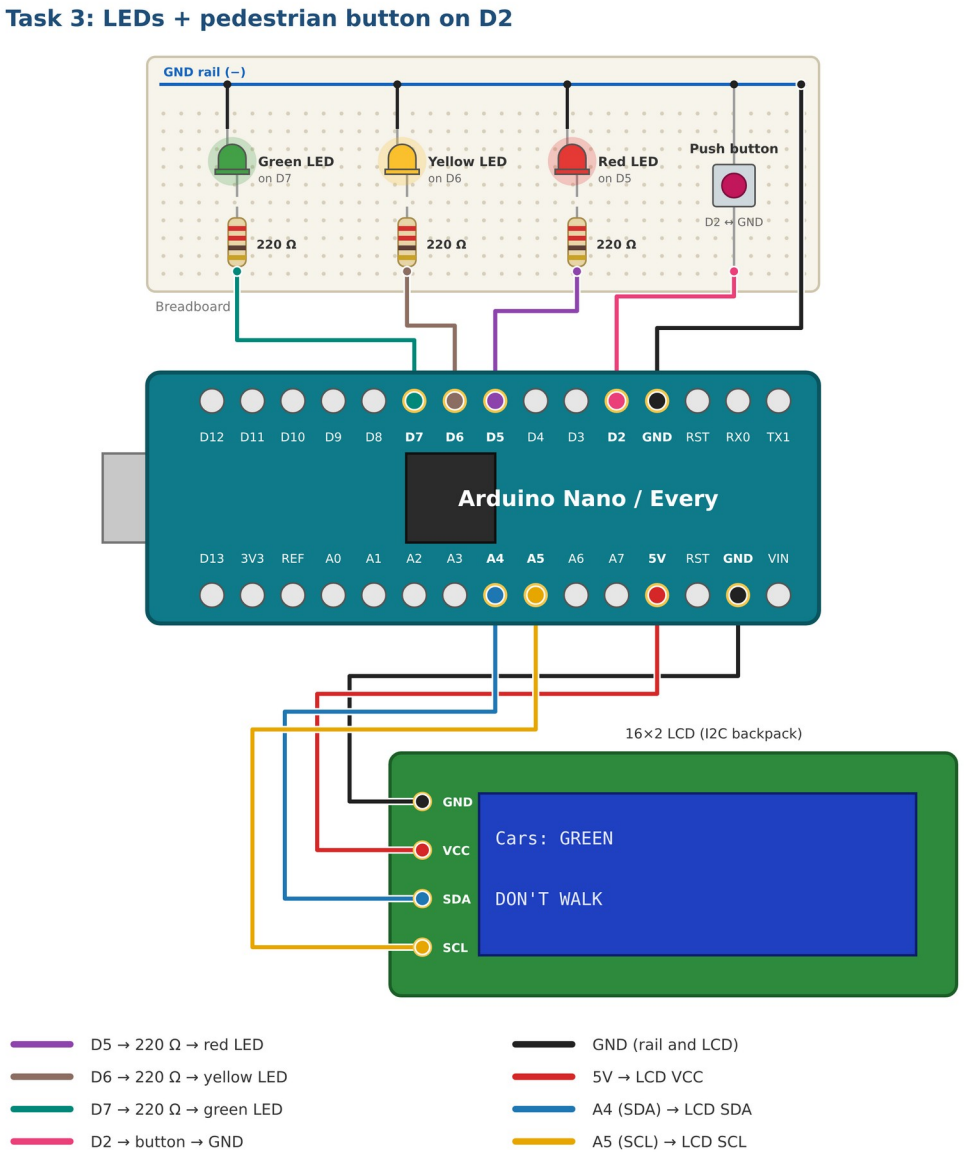


Figure 6: Task 3 wiring: the Task 2 circuit plus a push button between D2 and GND

No resistor is needed for the button: `pinMode(BUTTON_PIN, INPUT_PULLUP)` switches on a resistor inside the chip that holds D2 HIGH. Pressing the button connects D2 to GND, so it reads LOW while pressed.

Why there is no delay() in this sketch

During `delay(5000)` the board does nothing else for 5 seconds: it does not read any pins. A quick tap on the button in that time is simply missed. Task 3 never waits. Instead `loop()` runs thousands of times a second, and each time it checks the button and asks *has this state lasted long enough yet?*

© 2026 by University of Limerick. Author: Luke Griffin

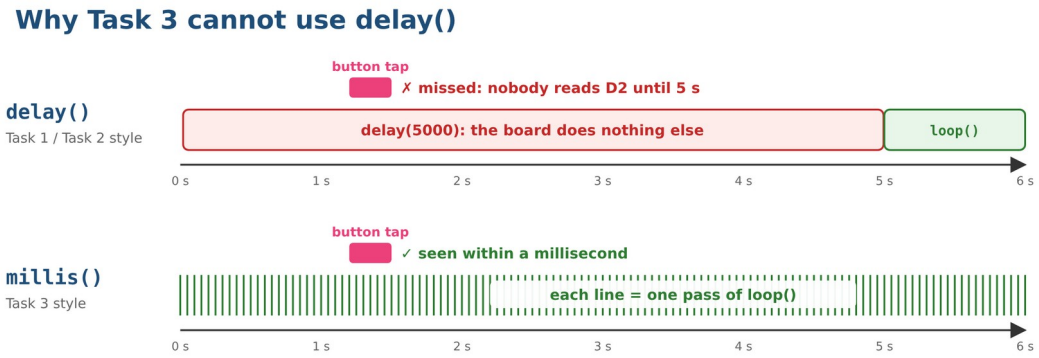


Figure 7: With delay() the board is blind while it waits; with millis() it checks everything on every pass

`millis()` is the number of milliseconds since the board started. The sketch saves it in `stateStart` whenever it enters a state, so `millis() - stateStart` is how long it has been in the current state. All the things that happen **once**, on the way into a state (note the time, set the LEDs, update the LCD), are in one function, `enterState()`.

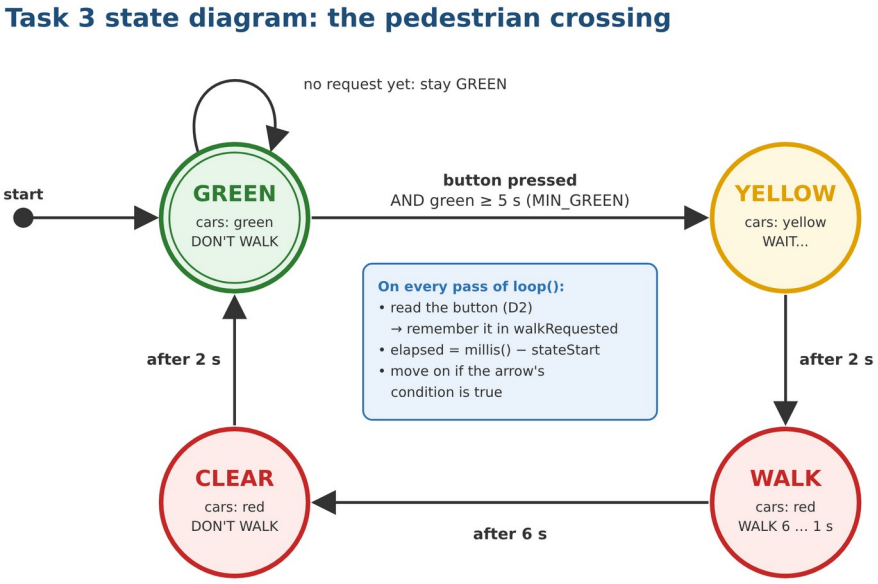


Figure 8: Task 3 state diagram. GREEN waits for two conditions: a request and the minimum green time.

State	Cars' LEDs	LCD bottom row	Leaves when...
GREEN	green	DON'T WALK (or WAIT... after a press)	button pressed AND ≥ 5 s in GREEN
YELLOW	yellow	WAIT...	2 s have passed
WALK	red	WALK 6s ... WALK 1s	6 s have passed
CLEAR	red	DON'T WALK	2 s have passed

© 2026 by University of Limerick. Author: Luke Griffin

What happens

> **Code**
>
> Upload `Task3_Pedestrian_Crossing` (with `RGB_SHIELD` set for your display) and open the Serial Monitor.

The light stays GREEN until you press the button. A press is remembered in `walkRequested`, so even a very short tap is enough, and the LCD shows `WAIT...`. If the light has already been GREEN for 5 s it changes straight away; otherwise it waits until the 5 s minimum is up, so the cars always get some green time.

Task 3 questions

3a Run it

Upload the code and press the button. **(record)** Paste the Serial Monitor output for one crossing. It should look like this:

Serial Monitor

```
Crossing ready
-> GREEN
Button pressed
-> YELLOW
-> WALK
-> CLEAR
-> GREEN
```

3b Press it at different times

Press the button at each of the times below and note what happens. Use the state diagram to explain each result.

When you press	What happens	Why
Just after the light turns GREEN		
When GREEN has been on for more than 5 s		
During WALK		
During CLEAR		

3c delay() would miss it

Imagine Task 2 with a button check added at the top of `loop()`. What is the longest time a pedestrian could have to hold the button before the board noticed? Why does Task 3 notice a tap of a few milliseconds?

Your answer:

© 2026 by University of Limerick. Author: Luke Griffin

3d No debouncing needed

Mechanical buttons 'bounce': one press can read as several quick presses. Why does that not cause a problem in this sketch? (Hint: what does a second press do to `walkRequested`?)

Your answer:

6 Task 4: A code lock with SHA-256

Objective: combine a state machine with the hashing from last week. You type a code into the Serial Monitor; the board works out its **SHA-256** hash and compares it with the hash stored in the sketch. The code itself is never stored anywhere. The wiring is the same as Task 3 (the button is not used).

SHA-256 (from last week)

Last week you used SHA-1, which turns any message into a 40-character fingerprint. SHA-256 works in the same way, but its fingerprint is longer and it is still considered secure. You only need to know it at this level: the same message always gives the same hash, a tiny change gives a completely different hash, and you cannot work backwards from a hash to the message.

	SHA-1 (Week 05)	SHA-256 (this week)
Hash size	160 bits	256 bits
Written as	40 hex characters	64 hex characters
Rounds per 64-byte block	80	64
Still safe for security?	No: two different messages with the same hash were found in 2017	Yes

SHA-256 of two codes that differ by one character

Code	SHA-256 hash
`ET4047`	`e0fe78d09229a737c8b66424c9b4bd456828da3b547128503a4cd41c6aa51fd1`
`ET4048`	`a41b9b9eaf17c5f24161cd6af5b4dc37aa249e7d12223e22735f72872ece6747`

> **Want to see inside SHA-256?**
>
> You do not need to know how the rounds work. If you are curious, this step-by-step example walks through one hash on an Arduino: forum.arduino.cc/t/a-step-by-step-sha256-data-hashing-updated-with-full-source-code-external-source/631262. The `sha256Hex()` function at the bottom of the Task 4 sketch is commented in the same way as last week's SHA-1 code.

© 2026 by University of Limerick. Author: Luke Griffin

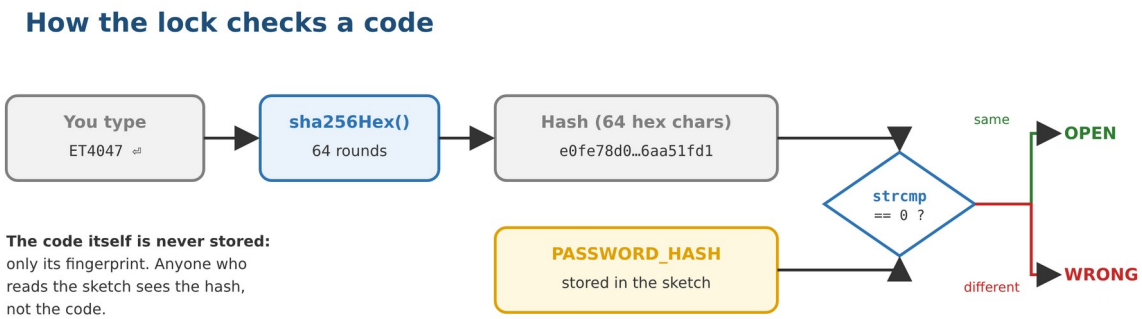


Figure 9: The lock never compares codes, only hashes

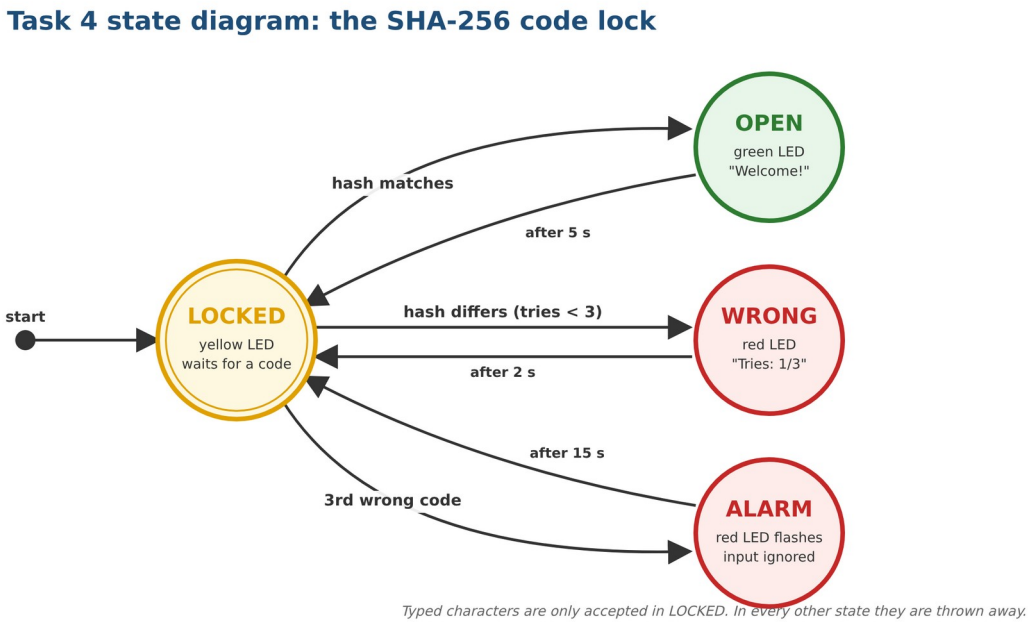


Figure 10: Task 4 state diagram. Three wrong codes in a row trigger the ALARM state.

What happens

> **Code**
>
> Upload `Task4_SHA256_Lock` (with `RGB_SHIELD` set for your display). Open the Serial Monitor at 9600 baud and set the line ending (bottom right) to **Newline**.

The lock starts in LOCKED with the yellow LED on. Type a code and press Enter. The board prints what you typed and its hash. If the hash matches `PASSWORD_HASH` the lock goes to OPEN (green LED) for 5 s. If not, it goes to WRONG (red LED) for 2 s. After three wrong codes in a row it goes to ALARM: the red LED flashes and everything you type is ignored for 15 s. The code is `ET4047`.

Task 4 questions

4a Run it

Type a wrong code, then `ET4047`. **(record)** Paste the Serial Monitor output. It should look like this:

Serial Monitor

```
Code lock ready. Type the code and press Enter.
-> LOCKED
Typed:   1234
SHA-256: 03ac674216f3e15c761ee1a5e255f067953623c8b388b4459e13f978d7c846f4
-> WRONG
-> LOCKED
Typed:   ET4047
SHA-256: e0fe78d09229a737c8b66424c9b4bd456828da3b547128503a4cd41c6aa51fd1
-> OPEN
```

4b Trigger the alarm

Type three wrong codes in a row, then type `ET4047` straight away while the red LED is flashing. What happens, and why? Which line of the sketch makes this happen?

Your answer:

4c Your own code

Choose your own code. Type it while the sketch is running, copy the `SHA-256:` line from the Serial Monitor into `PASSWORD_HASH`, and upload again. **(record)** Show that your code opens the lock and `ET4047` no longer does.

Your answer:

4d Questions

1. `ET4047` and `ET4048` differ by one character. Compare their hashes in the table above. How many of the 64 hex characters are the same?

2. Anyone can read the sketch and see `PASSWORD_HASH`. Why does that not tell them the code?

3. A 4-digit PIN has 10 000 possible values. With the ALARM state (15 s after every 3 wrong tries), roughly how long would it take to try them all? Why does this make the ALARM state useful?

4. If the code were a single digit, could someone find it from the hash? What does that tell you about short codes?

© 2026 by University of Limerick. Author: Luke Griffin

Your answer:

© 2026 by University of Limerick. Author: Luke Griffin